



APRENDIZADO PROFUNDO

deep learning

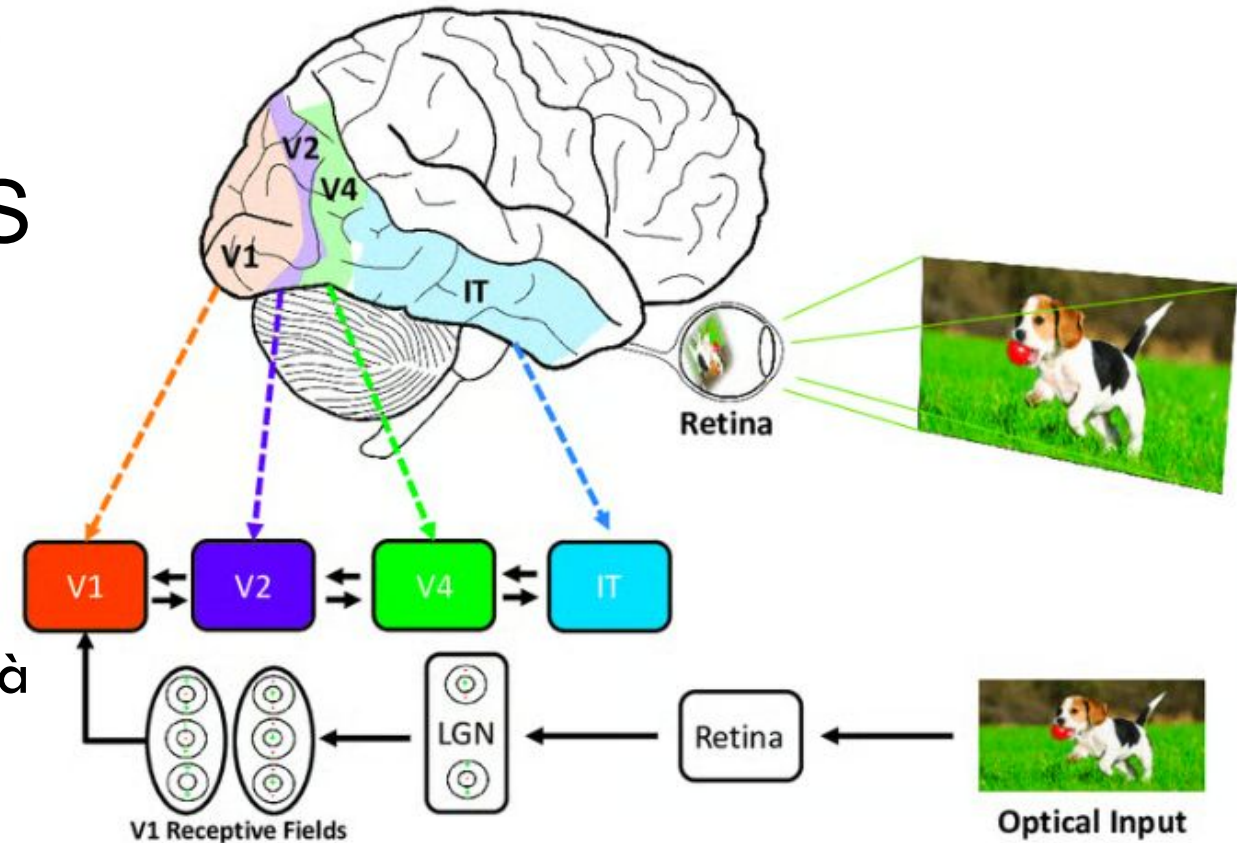
PROF. JOSENALDE OLIVEIRA

PROGRAMA DE PÓS-GRADUAÇÃO EM TI (PPgTI) - UFRN

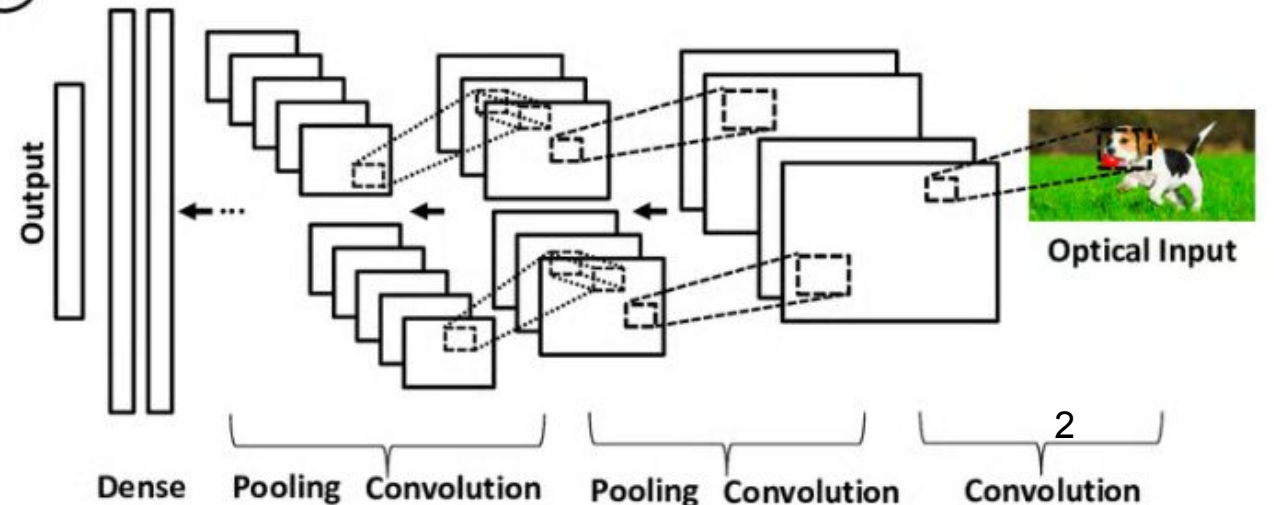
REDES CONVOLUCIONAIS (*CONVNET*) - córtex visual

- reconhecimento de objetos (classificação, detecção/localização, segmentação)
- extrair características de forma autônoma em grandes conjuntos de dados, invariante à posição, escala, orientação
- modelos pré-treinados facilitam transfer learning (EfficientNet, ResNet, MobileNet, VGG etc.)
- hierarquia de camadas, com aumento na captura de detalhes

a

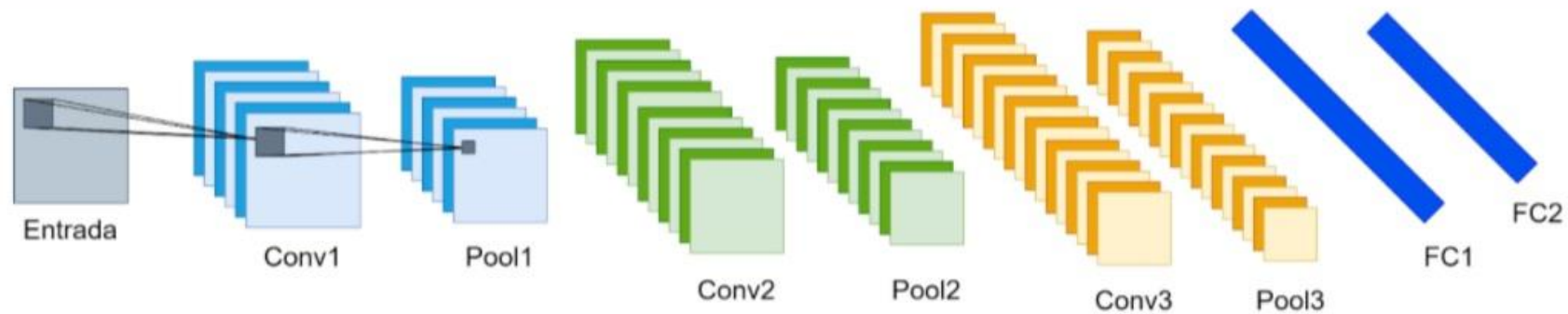


b



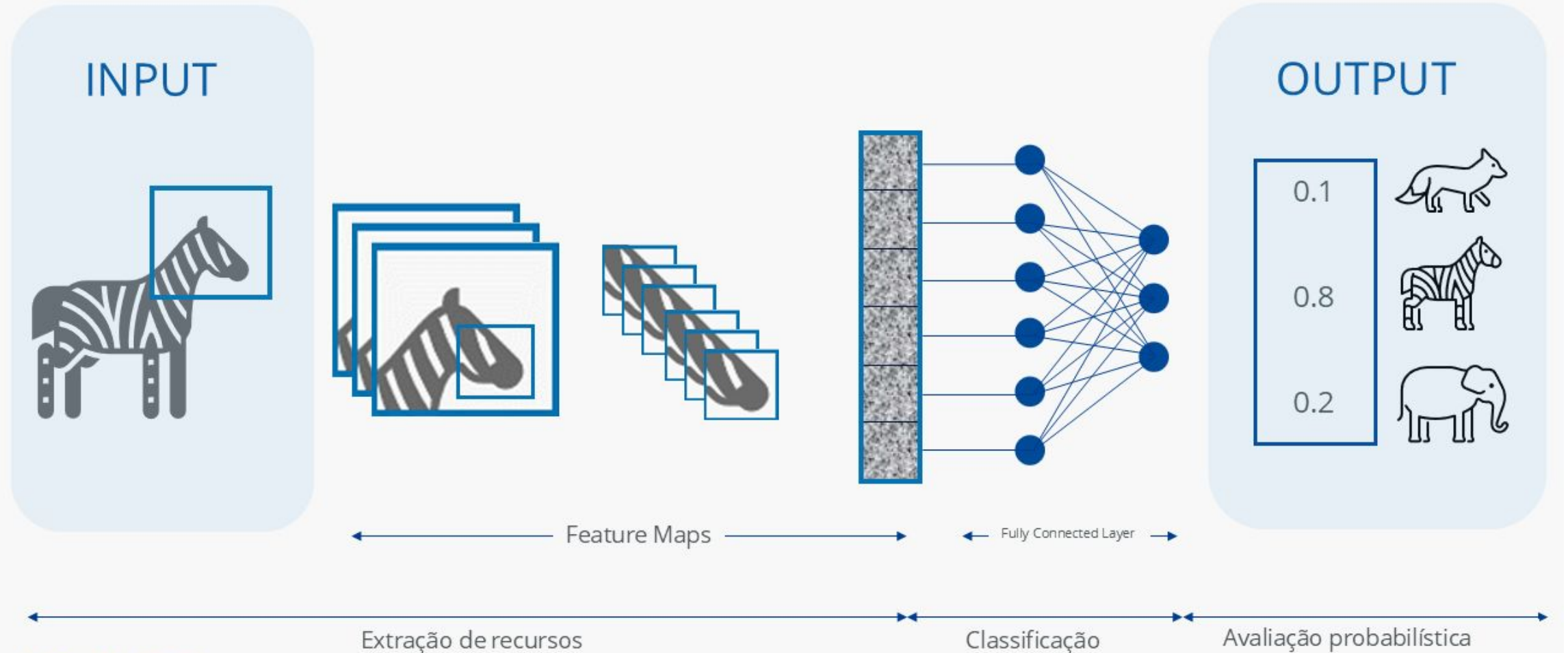
REDES CONVOLUCIONAIS

Aprendizado Hierárquico



- Múltiplas camadas produzem um **aprendizado hierárquico**
 - Primeiras camadas: características de baixo nível
 - Últimas camadas: características de alto nível

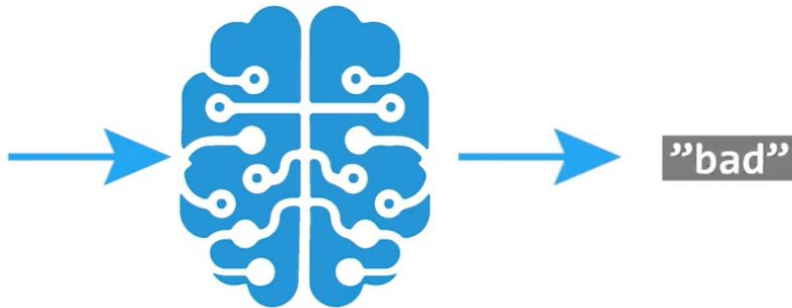
CONVOLUTIONAL NEURAL NETWORK (CNN)



REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

<https://www.mvtec.com/company/research/datasets/mvtec-ad>

Deep-learning-based anomaly detection
What is it?



Deep-learning-based anomaly detection
What is it?



REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

<https://www.mvtec.com/company/research/datasets/mvtec-ad>

Deep-learning-based anomaly detection

What is it?

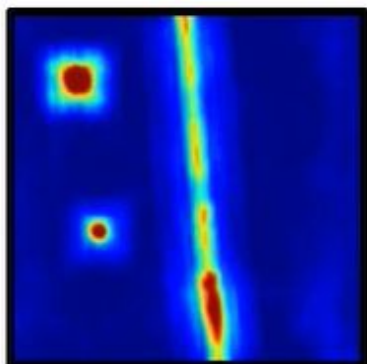


”good”



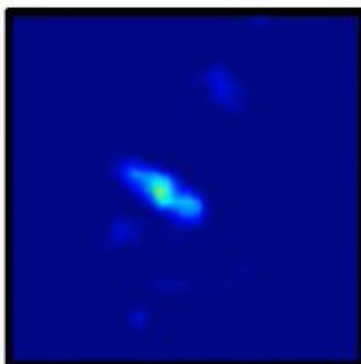
Input Image

HALCON



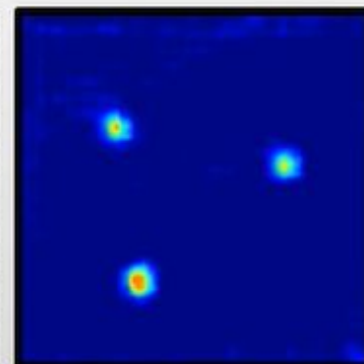
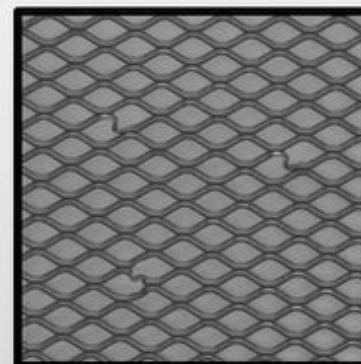
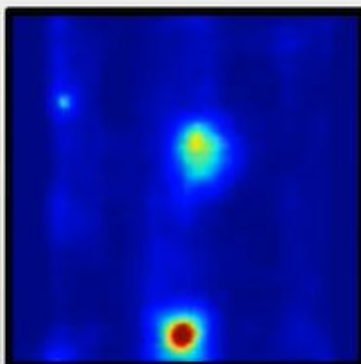
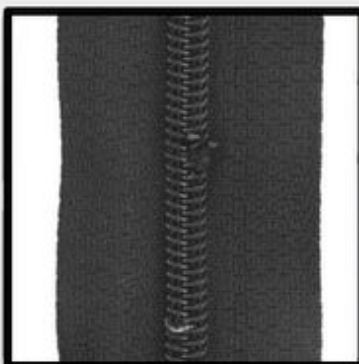
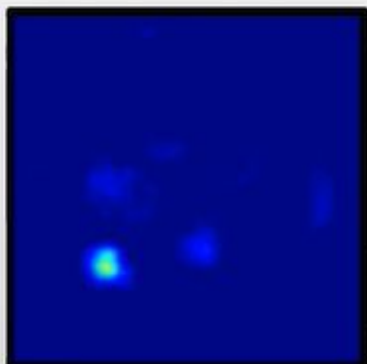
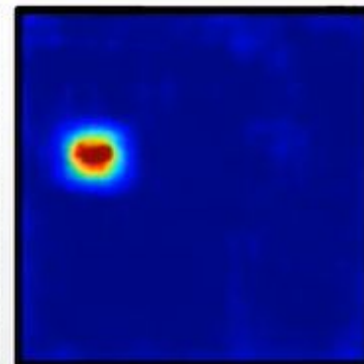
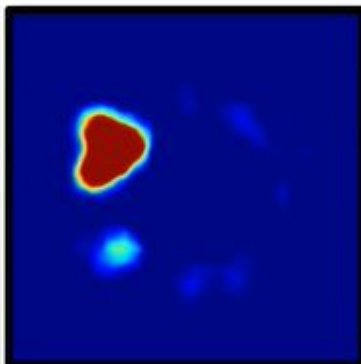
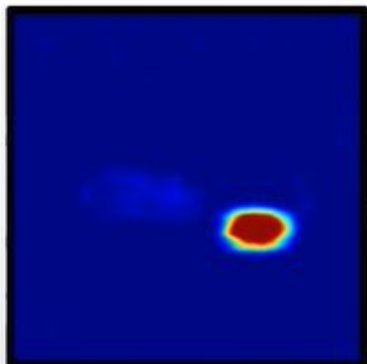
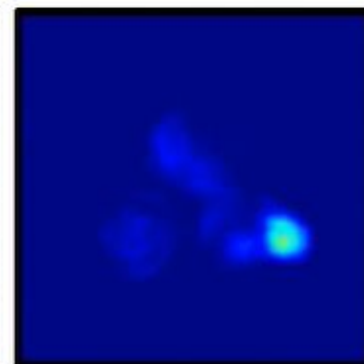
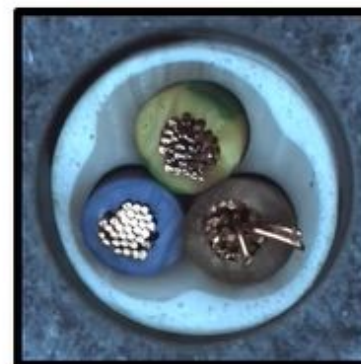
Input Image

HALCON



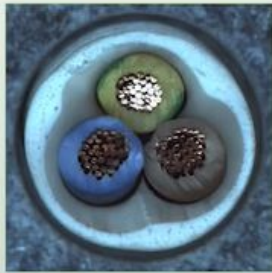
Input Image

HALCON

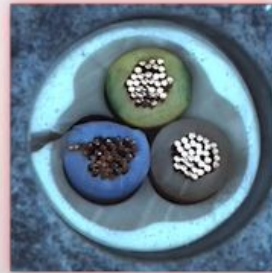


REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

Normal

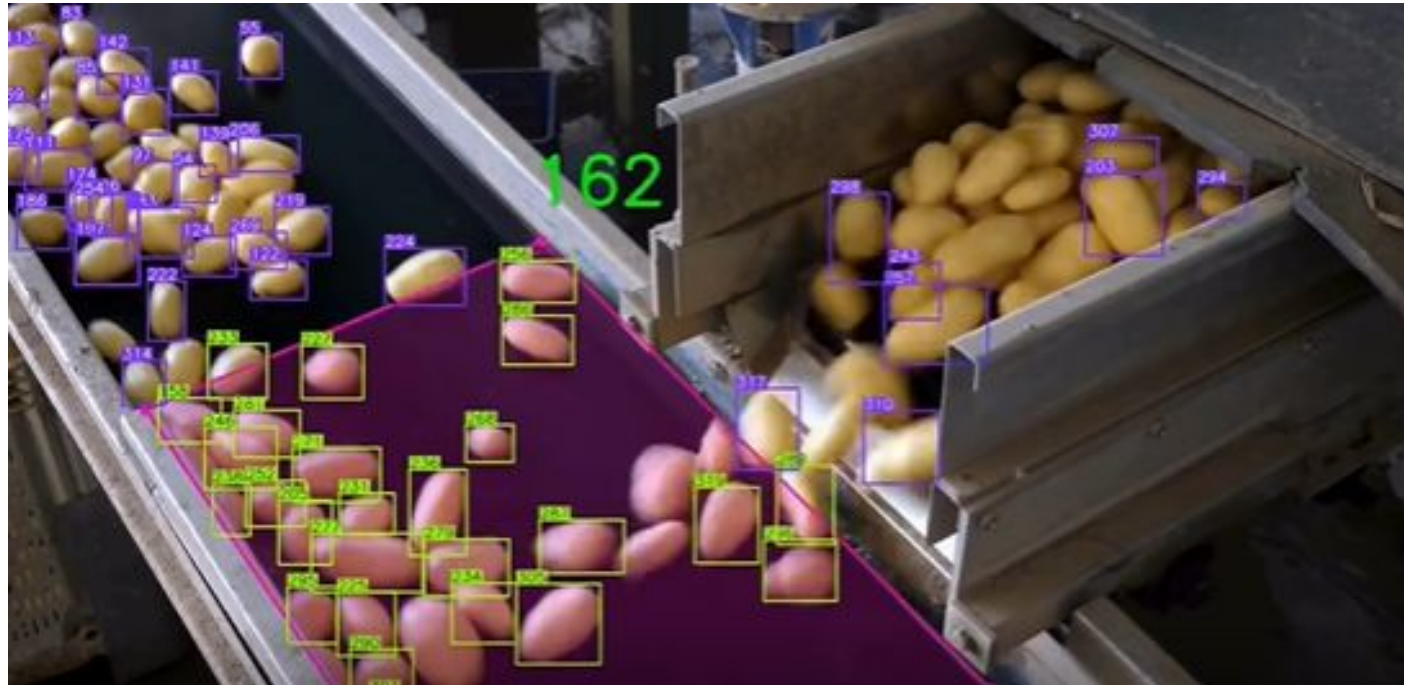


Anomalies



REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

<https://www.linkedin.com/feed/update/urn:li:activity:7396478479845978113/>



YOLO-nano (CNN especializada - bom para vídeos)

REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

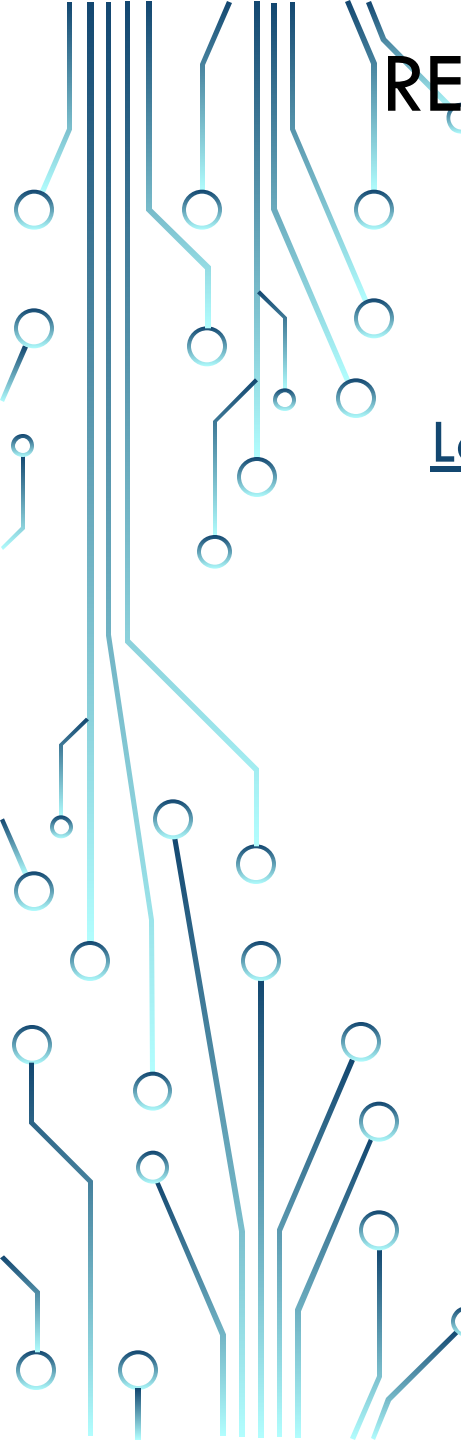
<https://www.linkedin.com/feed/update/urn:li:activity:7396478479845978113/>

Segment Anything Model - META

SAM 2
(applied to video)



<https://ai.meta.com/sam2/>



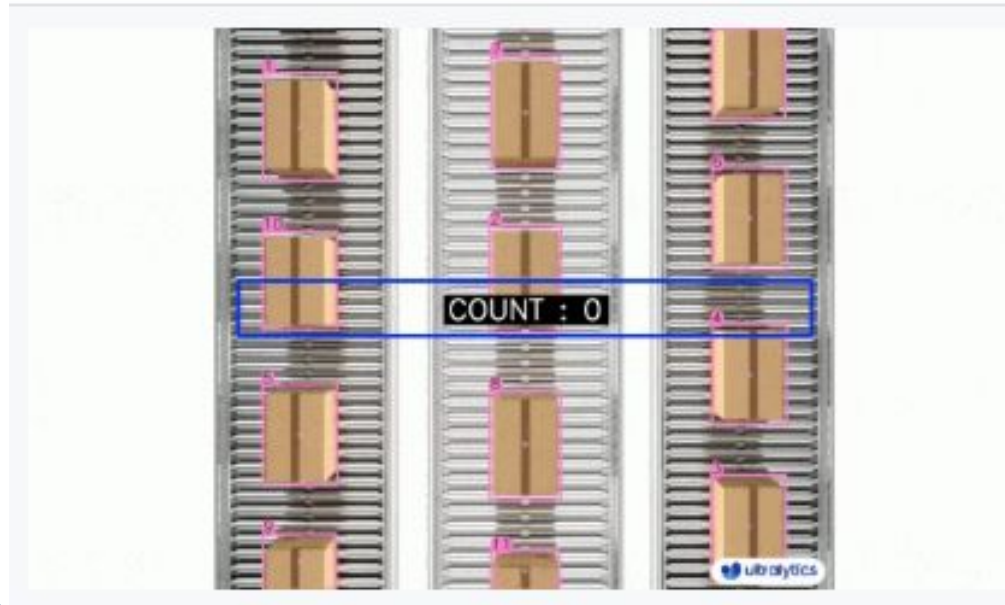
REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

Ler sobre técnicas de aprendizado N-Shot e arquiteturas mais usadas

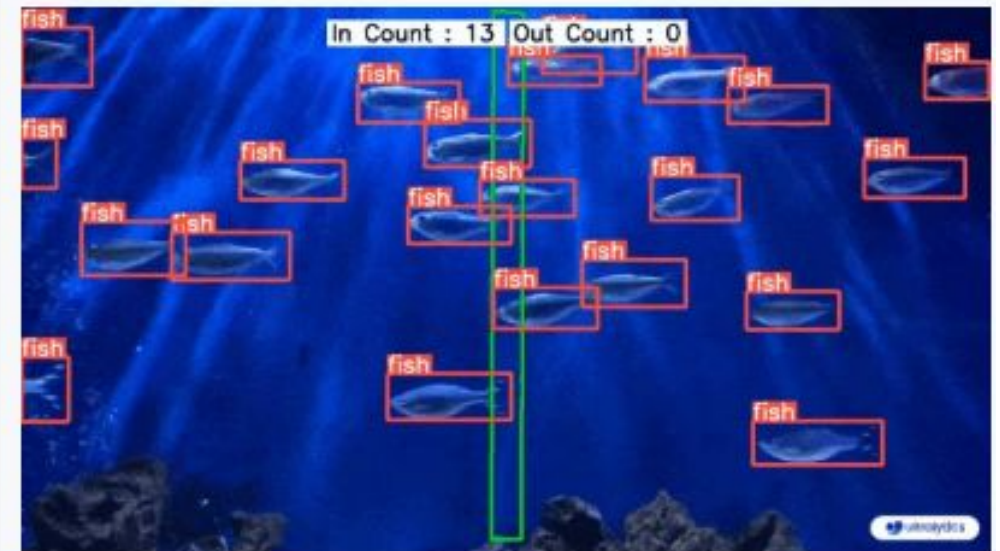
REDES CONVOLUCIONAIS - EXEMPLOS DE APLICAÇÕES

Object Counter - ULTRALYTICS

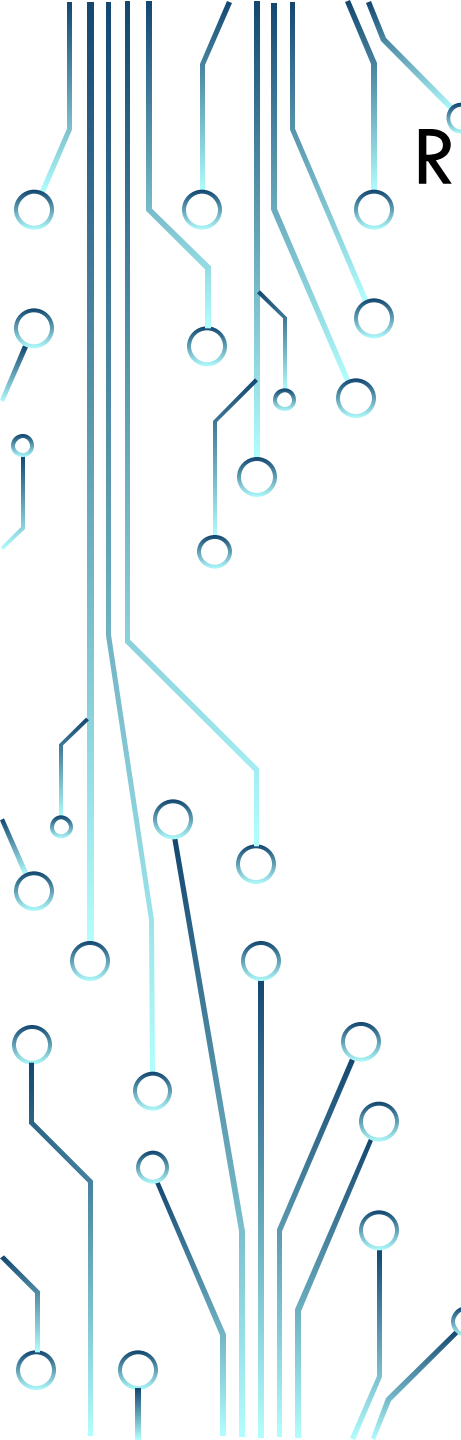
Contagem de Pacotes na Esteira Transportadora Usando Ultralytics YOLO11



Contagem de Peixes no Mar usando Ultralytics YOLO11



<https://docs.ultralytics.com/pt/guides/object-counting/>

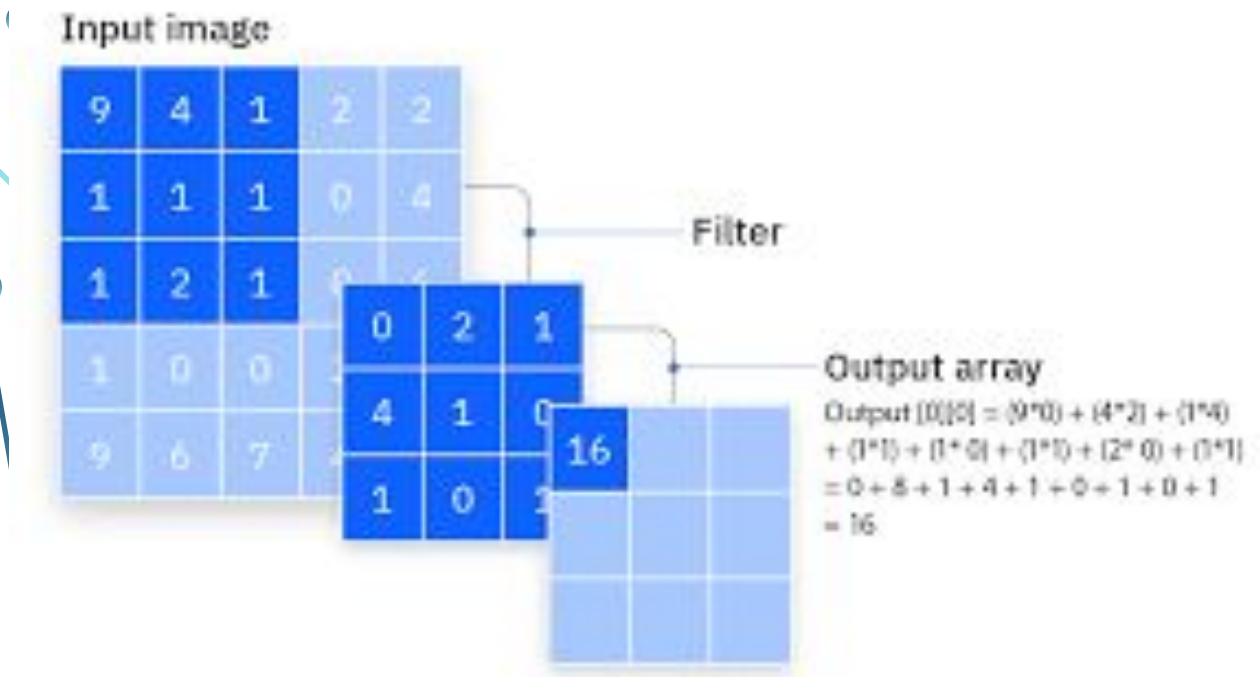


REDES CONVOLUCIONAIS

- LeCun et al. (1998) introduziram a LeNet-5, para além das camadas densas e funções sigmóides, as camadas **convolucionais** e as camadas de **pooling** (agrupamento)
- Suponha uma imagem $100 \times 100 \text{ px} = 10\text{kpx}$. Imagine uma camada com 1k neurônios. Já são 10.000×1.000 , 10 milhões de pesos. As CNNs resolvem usando camadas parcialmente conectadas e compartilhamento de peso na **mesma camada convolucional**
- As CNNs recebem as imagens com sua altura, largura e canais. Apenas na transição para camadas densas é que é necessário achatar a entrada
- Seus elementos são as **camadas convolucionais** com seus **filtros** para gerar mapas de características e as camadas de **pooling** para redução de dimensionalidade
- o treino das CNNs exige bastante RAM, pois o reverse do backpropagation exige os valores intermediários calculados no forward

REDES CONVOLUCIONAIS

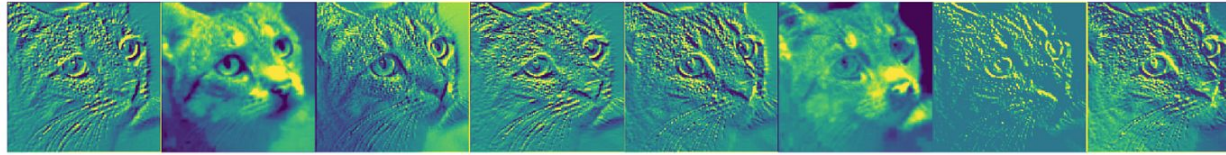
- o campo receptivo na CNN é o filtro ou kernel de convolução
- a mudança de um campo receptivo se chama stride (deslocamento)
- DNN normal aprende a reconhecer algo em um local, só reconhece neste local. **CNN aprende e reconhece em qualquer local**



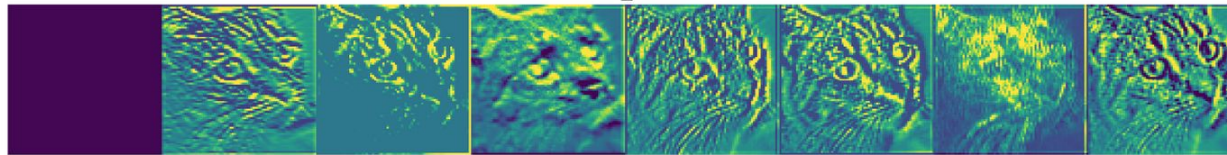
- uma camada de neurônios que utiliza o mesmo filtro gera um mapa de características que resalta as áreas de uma imagem que mais ativam o filtro (aprendido)
- pode ter N filtros, gerando um mapa por filtro - no mapa M, **todos os neurônios compartilham W e b**. A rede tem um neurônio por pixel em cada mapa
- detectam características diferentes

REDES CONVOLUCIONAIS - filtros e mapas

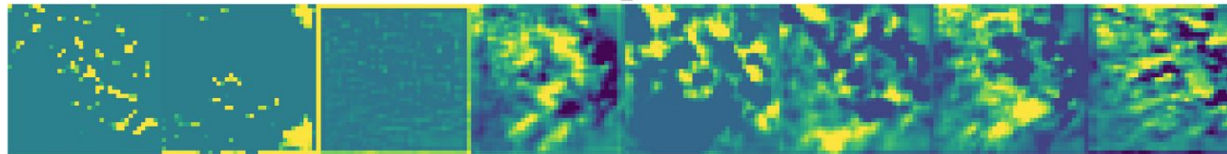
block1_conv1



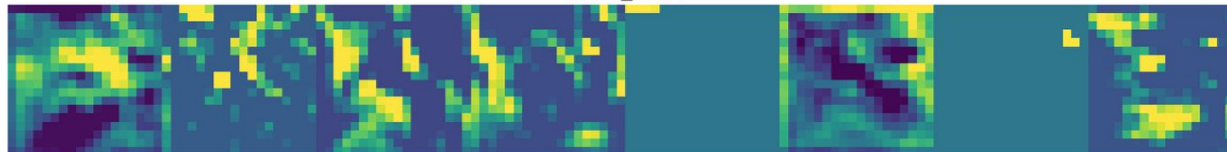
block2_conv1



block3_conv1



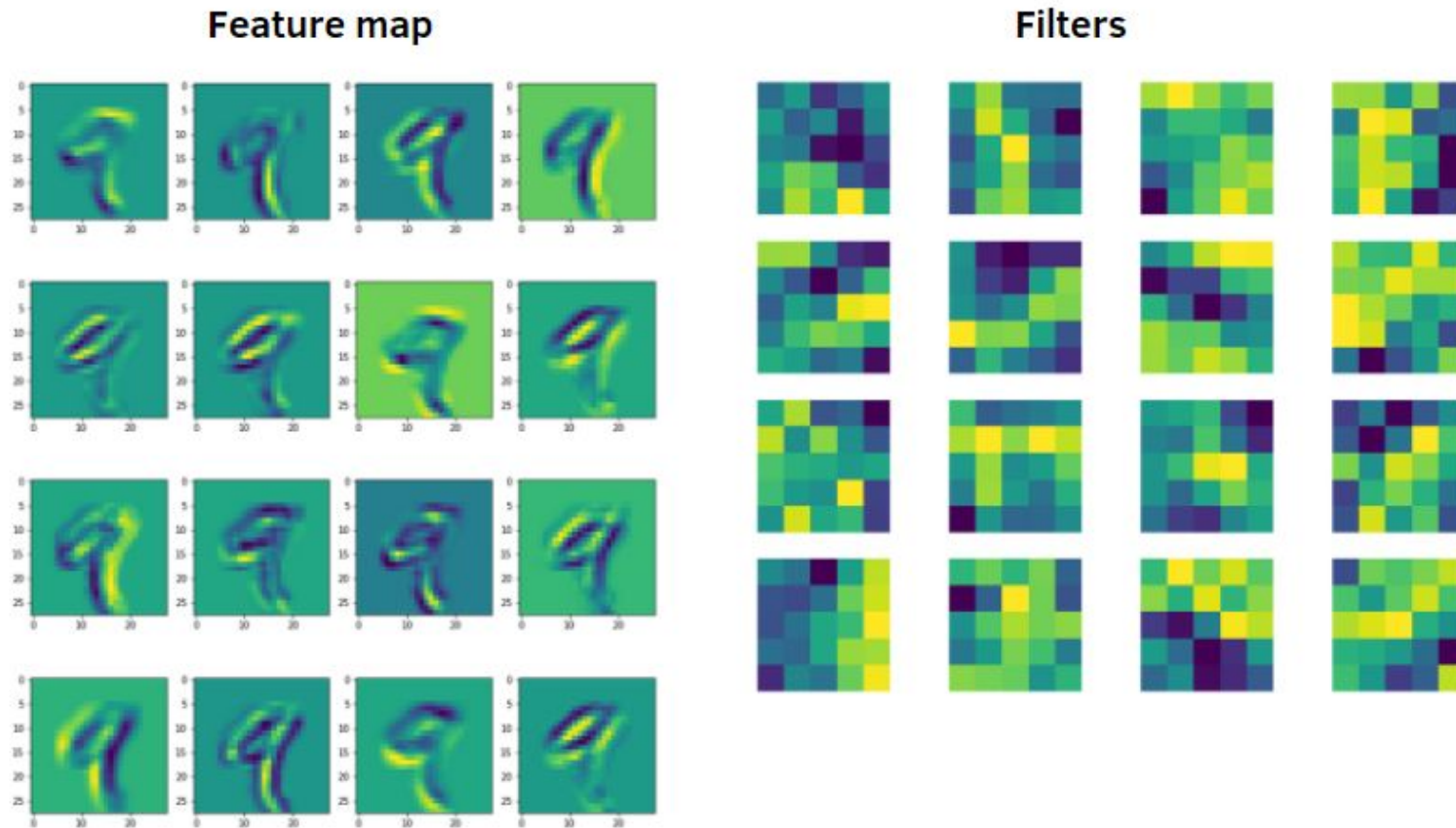
block4_conv1



block5_conv1

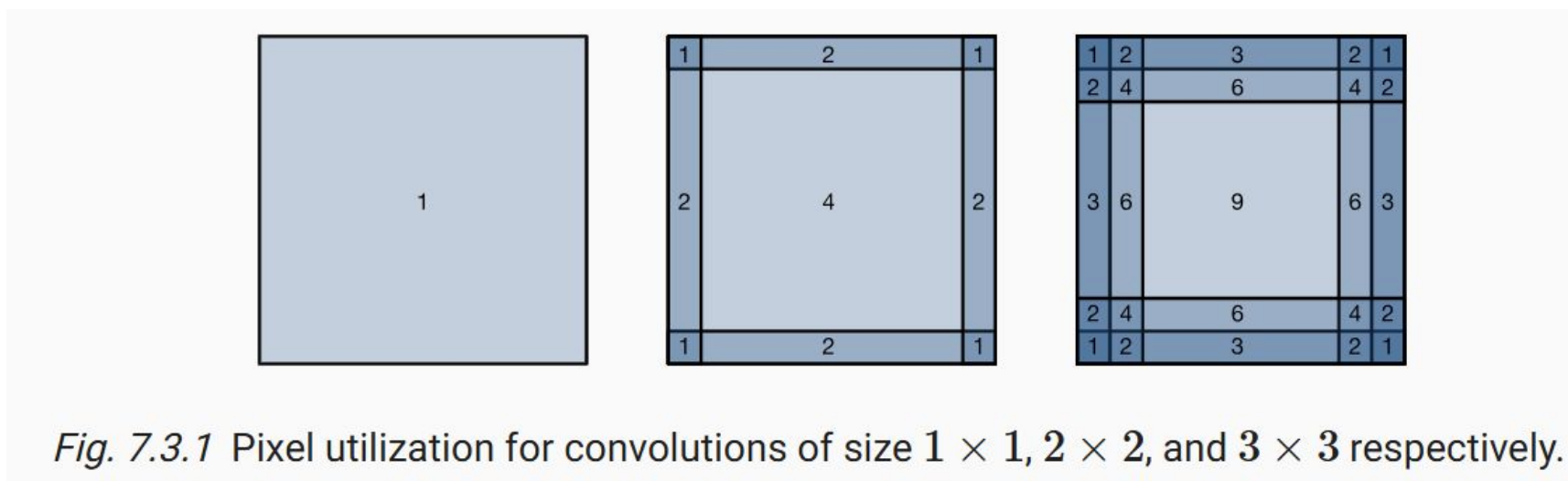


REDES CONVOLUCIONAIS - filtros e mapas



REDES CONVOLUCIONAIS - padding

- nas operações de convolução pode-se perder pixels nas bordas
- técnica preencher com zeros para aumentar tamanho efetivo da imagem

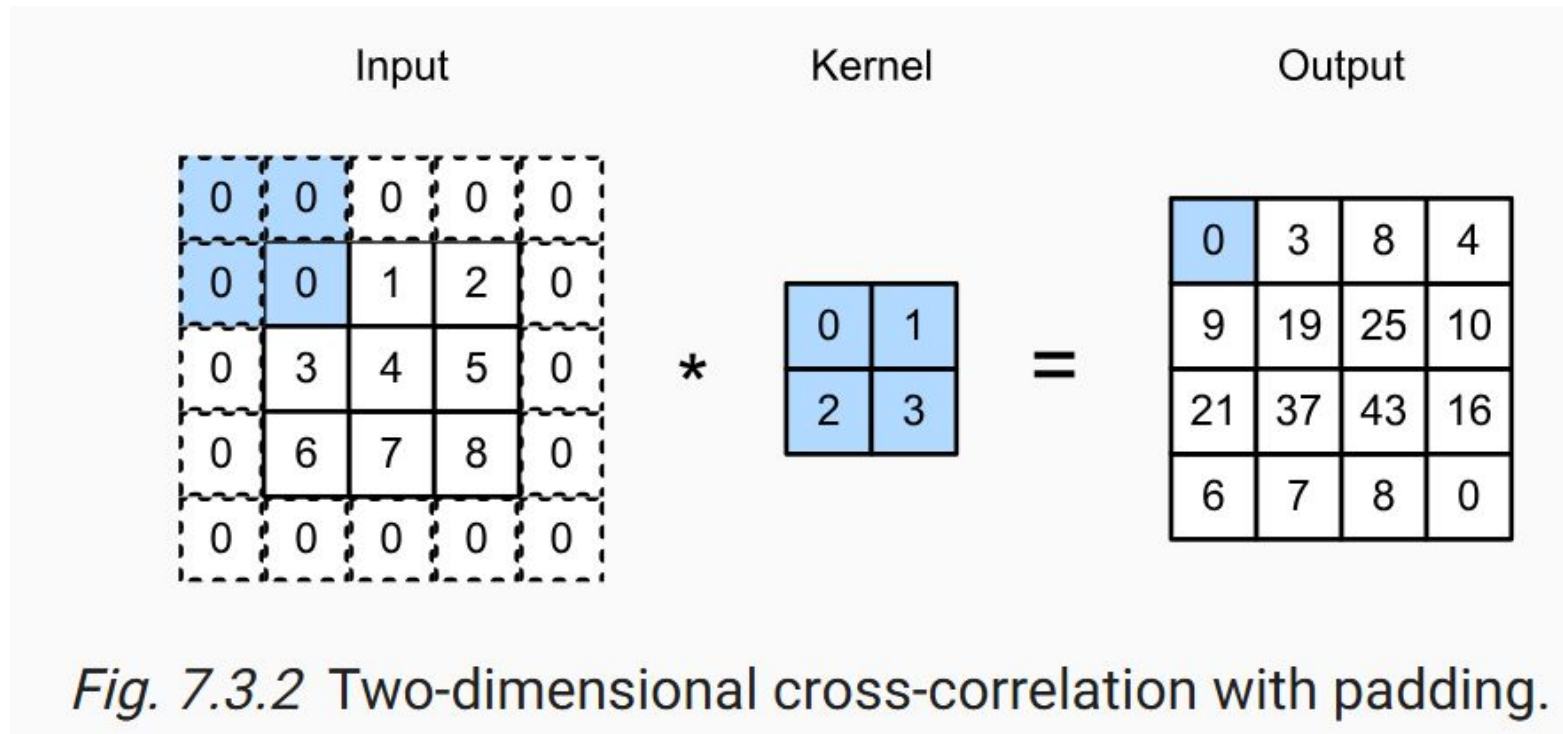


Fonte: [\[Zhang et al.\]](#)

same: Quando se adiciona padding suficiente para que a saída tenha as mesmas dimensões da entrada. Isso é útil para preservar as dimensões ao longo das camadas da rede e pode ser feito calculando o número necessário de pixels de padding para garantir que o filtro se aplique a todas as regiões da imagem.

REDES CONVOLUCIONAIS - padding

- usar kernels ímpares (1,3,5,7) com o padding igual left,right,bottom,top manter dimensionalidade
- No exemplo, a saída é $(nh - kh + ph + 1) \times (nw - kw + pw + 1)$



Fonte: [\[Zhang et al.\]](#)

REDES CONVOLUCIONAIS - camada pooling (agregação)

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

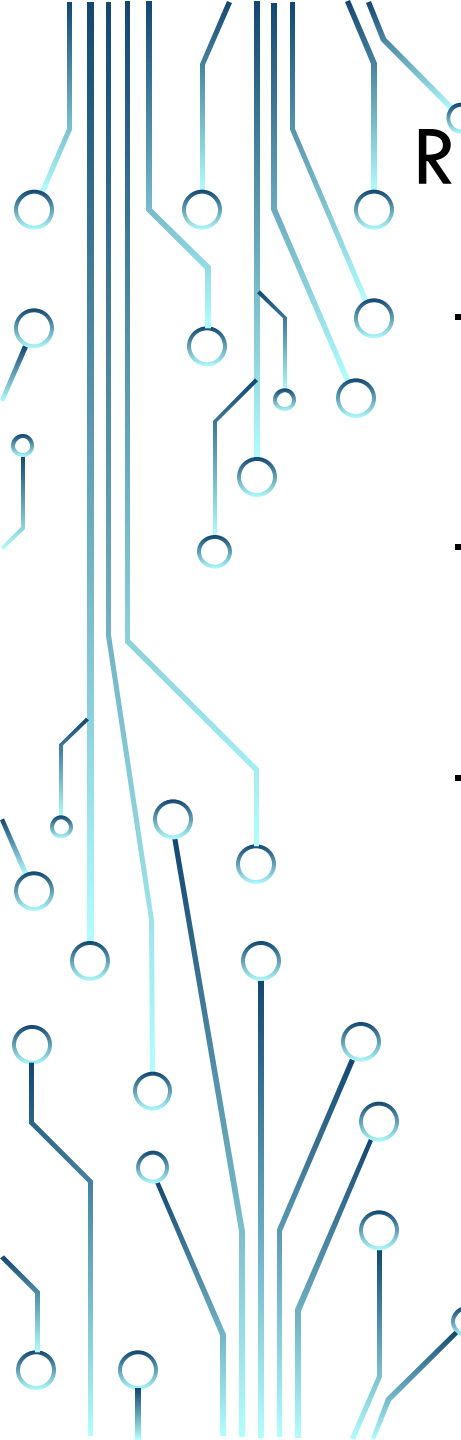
Max Pool



Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

- neurônio de pooling não tem pesos
- objetivo reduzir entrada, custo computacional/memória e parâmetros, reduzir sobreajuste
- funciona em cada canal independente

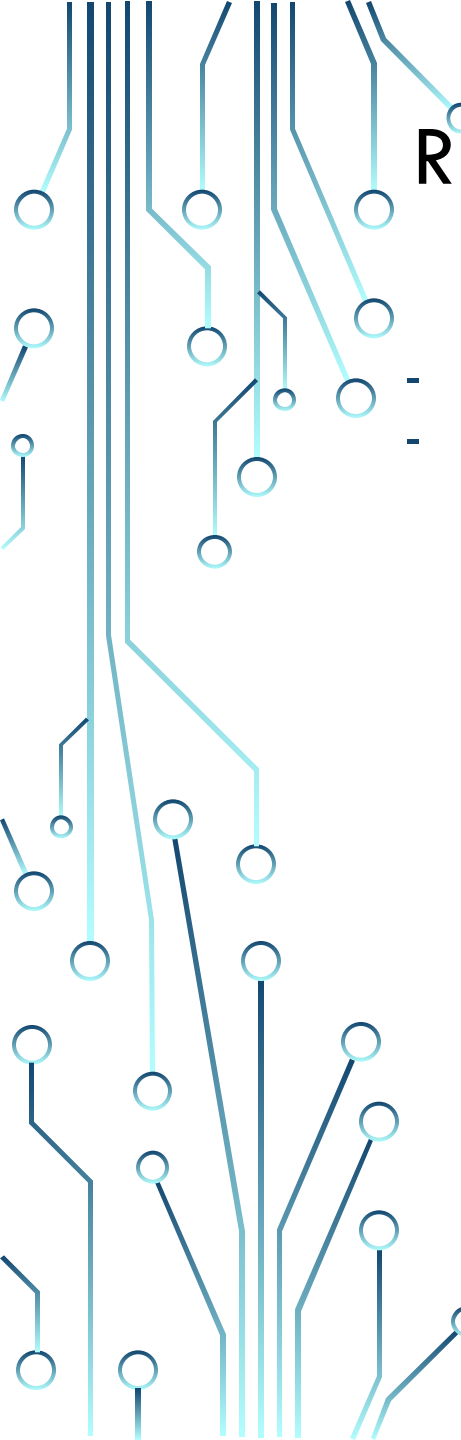


REDES CONVOLUCIONAIS - dicas práticas

- a primeira camada pode ter um kernel grande (por exemplo 5x5 com stride 2 ou mais) para reduzir a dimensão espacial da imagem sem perder muitas informações
- já nas demais camadas é melhor ao invés de ter uma com 5x5, ter duas com 3x3, usará menos parâmetros e exigirá menos cálculos e em geral, melhor desempenho
- é comum aumentar o número de filtros à medida que escalamos a CNN (64, 128, 256, etc.) - dobra-se o número de filtros após cada pooling, pois este divide espacialmente por fator 2, assim podemos dobrar o número de mapas de características na próxima camada sem receios computacionais

REDES CONVOLUCIONAIS - dicas práticas

```
model = keras.model.Sequential([
    keras.layers.Conv2D(64, 7, activation='relu', padding="same", input_shape=[28,18,1]),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(128,3,activation='relu',padding="same"),
    keras.layers.Conv2D(128,3,activation='relu',padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Conv2D(256,3,activation='relu',padding="same"),
    keras.layers.Conv2D(256,3,activation='relu',padding="same"),
    keras.layers.MaxPooling2D(2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, actitavion='softmax')])
```

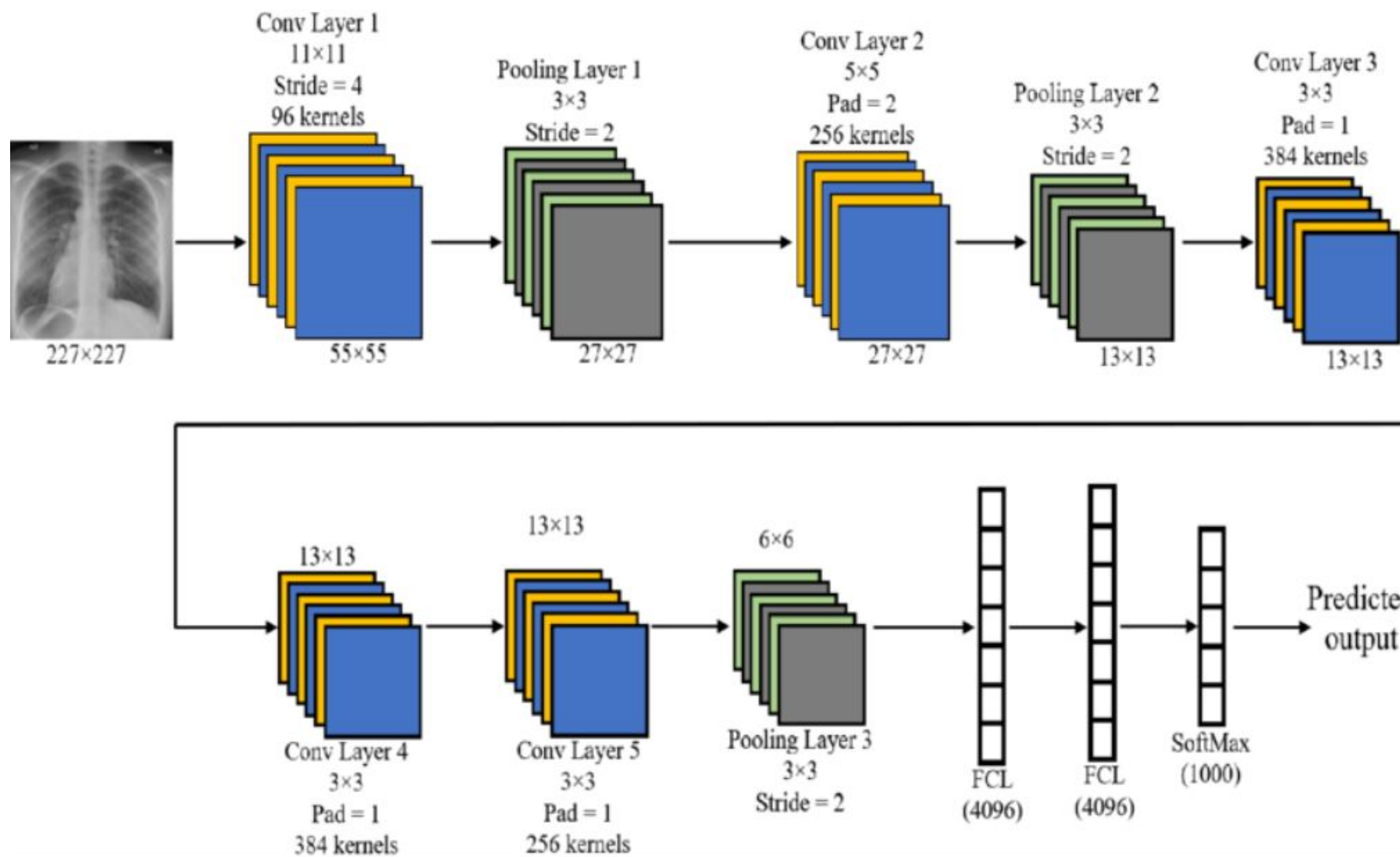


REDES CONVOLUCIONAIS -arquiteturas conhecidas

- LeNet-5 (1998)
- AlexNet (venceu ImageNet 2012) - empilha CNNs diretamente, sem necessariamente Pooling no topo da CNN. Usa técnica de Dropout (50%), data augmentation com deslocamentos, flipping horizontal e iluminação
 - GoogleNet (2015) tem 10x menos parâmetros que AlexNet. Usa subredes chamadas módulos inception (iniciação)
 - VGG (2014) (arquitetura simples)
 - ResNet (2015)
 - Xception, variante GoogleNet
 - SENet
 - EfficientNet (2019, Google) - menor que ResNet
 - MobileNet

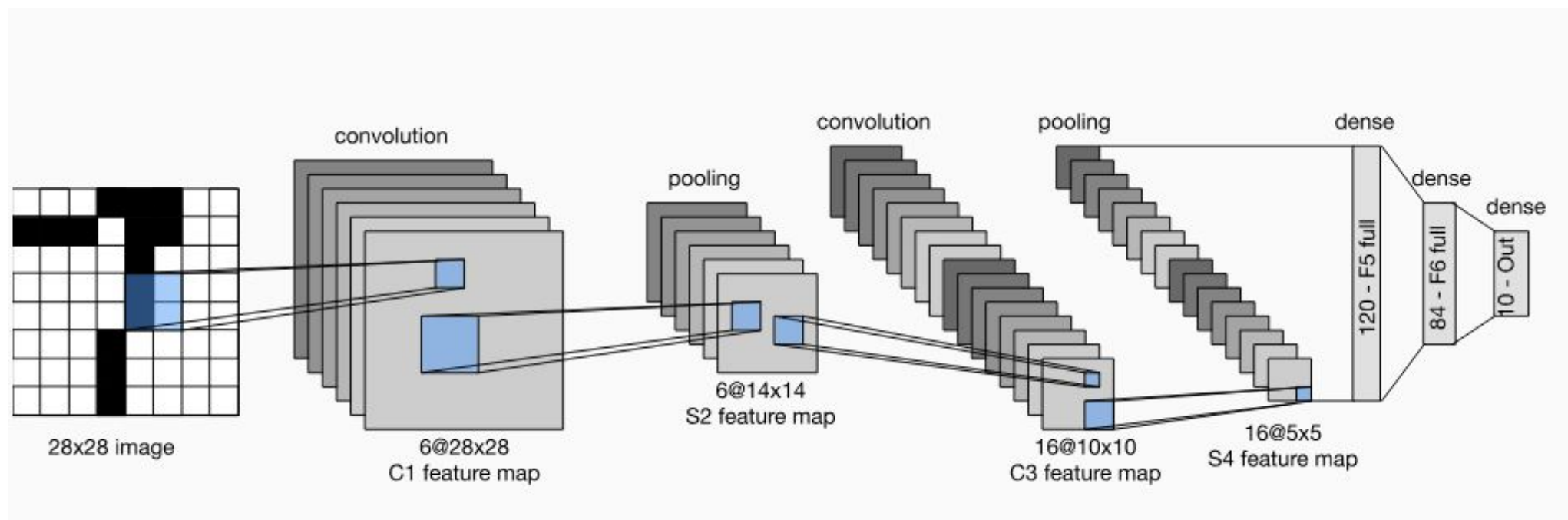
REDES CONVOLUCIONAIS -arquiteturas conhecidas

- AlexNet



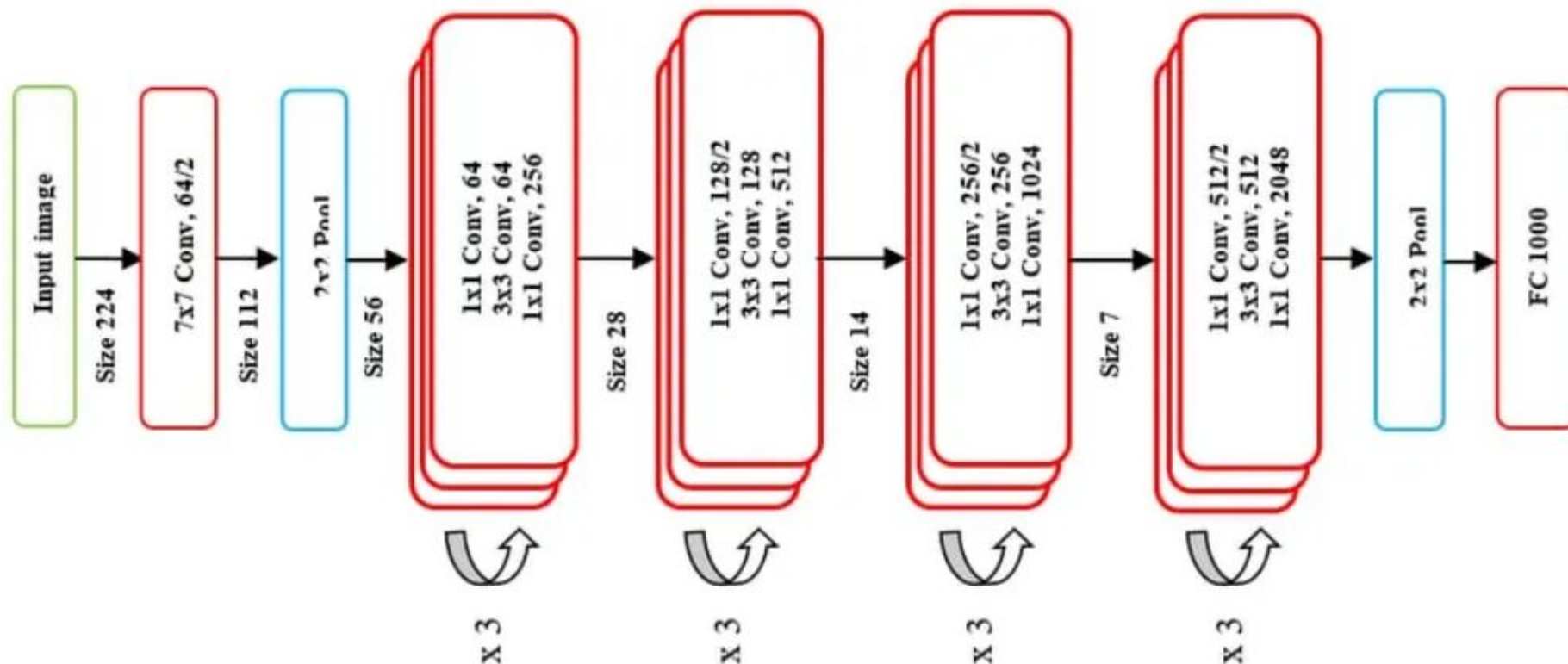
REDES CONVOLUCIONAIS -arquiteturas conhecidas

- [LeNet-5 \(1998\)](#)



REDES CONVOLUCIONAIS -arquiteturas conhecidas

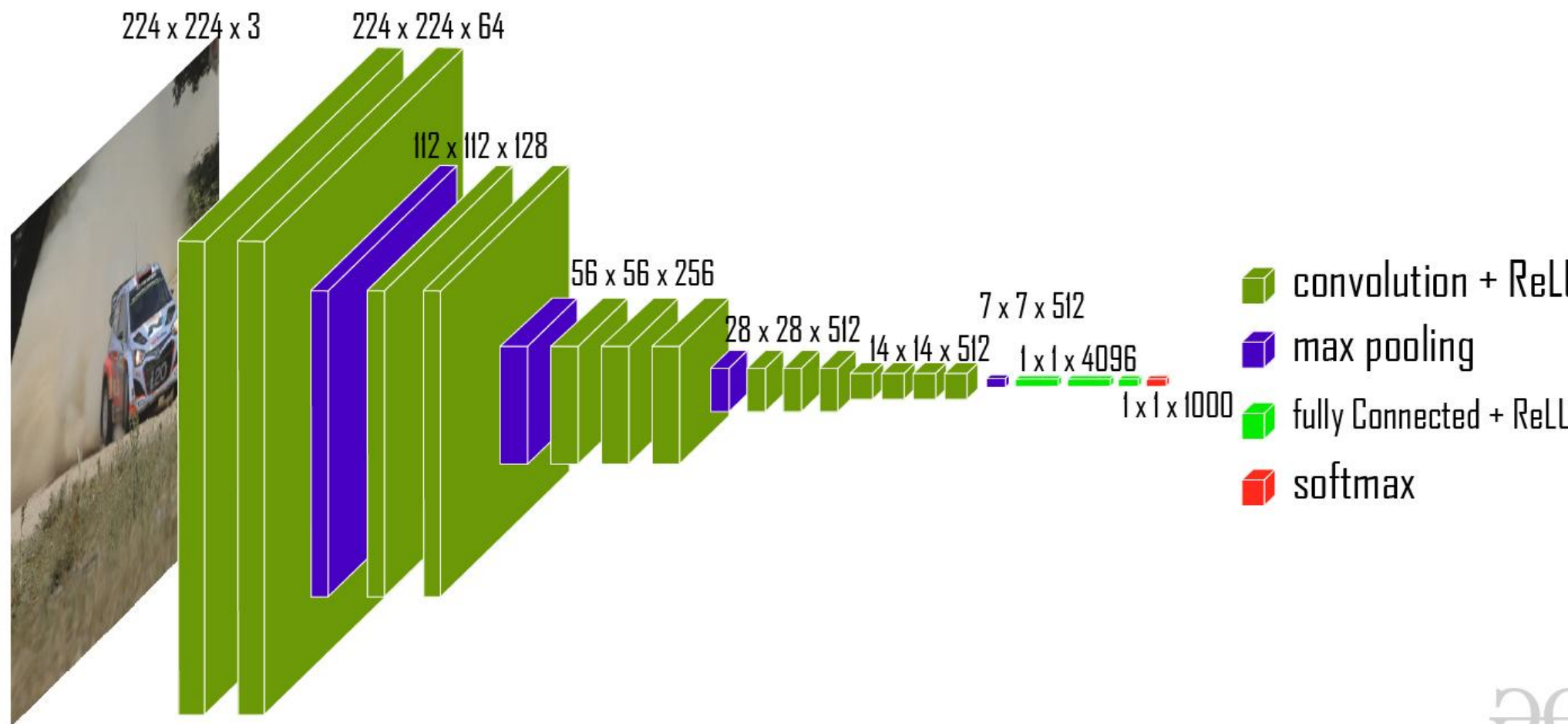
- ResNet Residual



ResNet-50 architecture

REDES CONVOLUCIONAIS -arquiteturas conhecidas

- VGG



REDES CONVOLUCIONAIS -arquiteturas conhecidas

- GoogleNet (Inception)

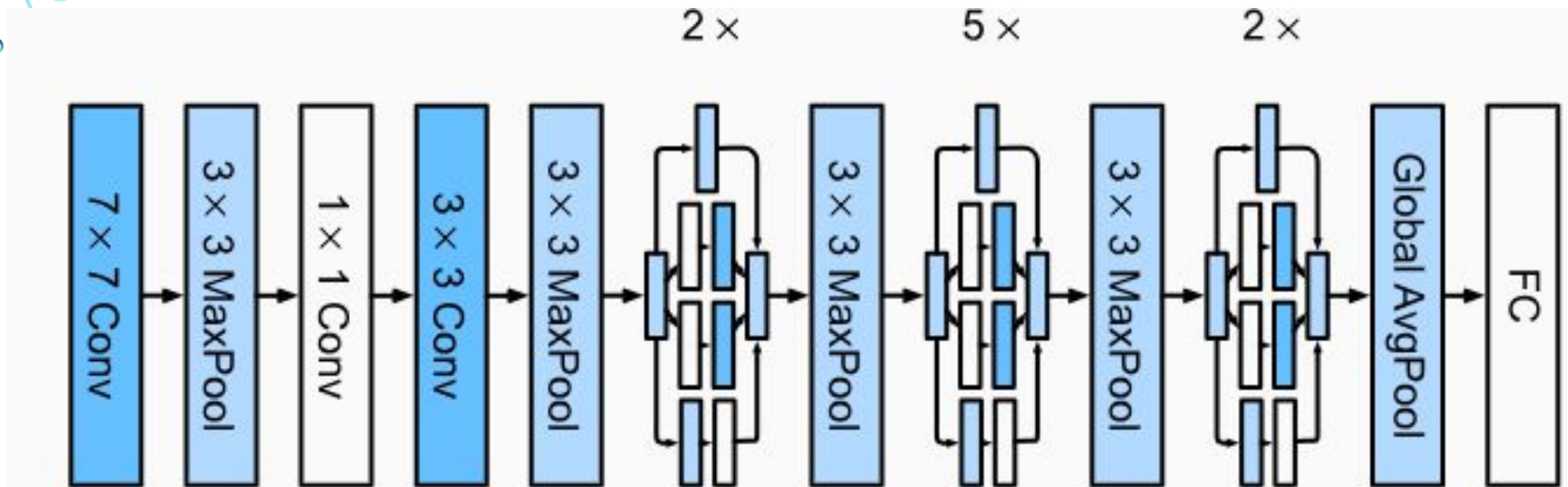


Fig. 8.4.2 The GoogLeNet architecture.

REDES CONVOLUCIONAIS -arquiteturas conhecidas

- GoogleNet (Inception)

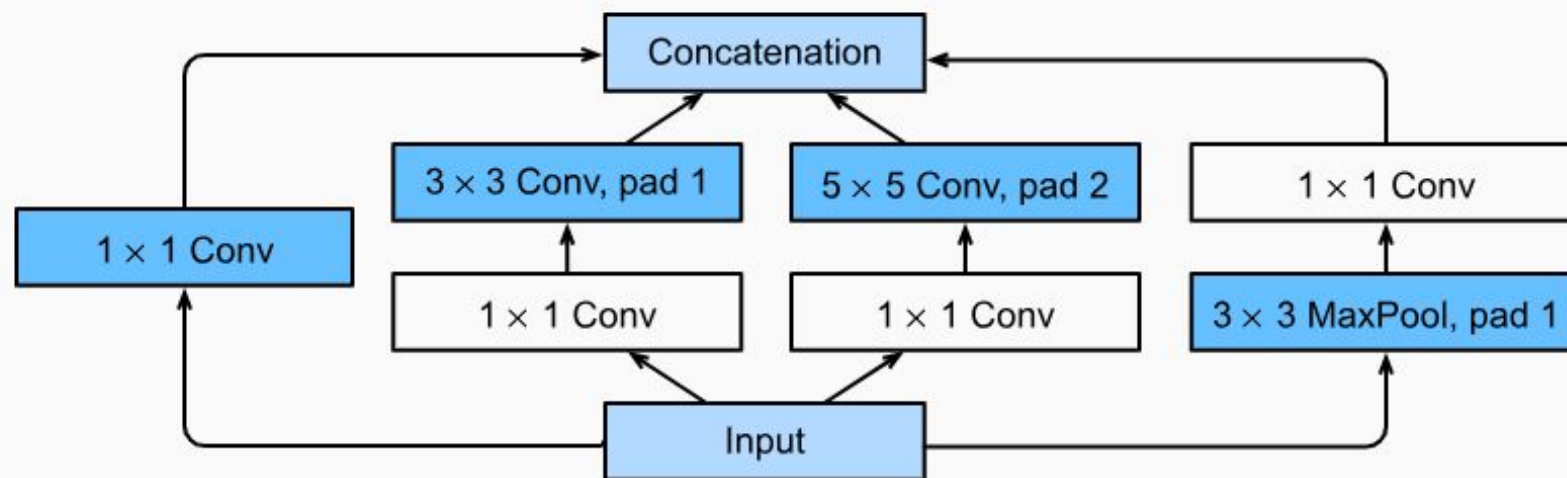
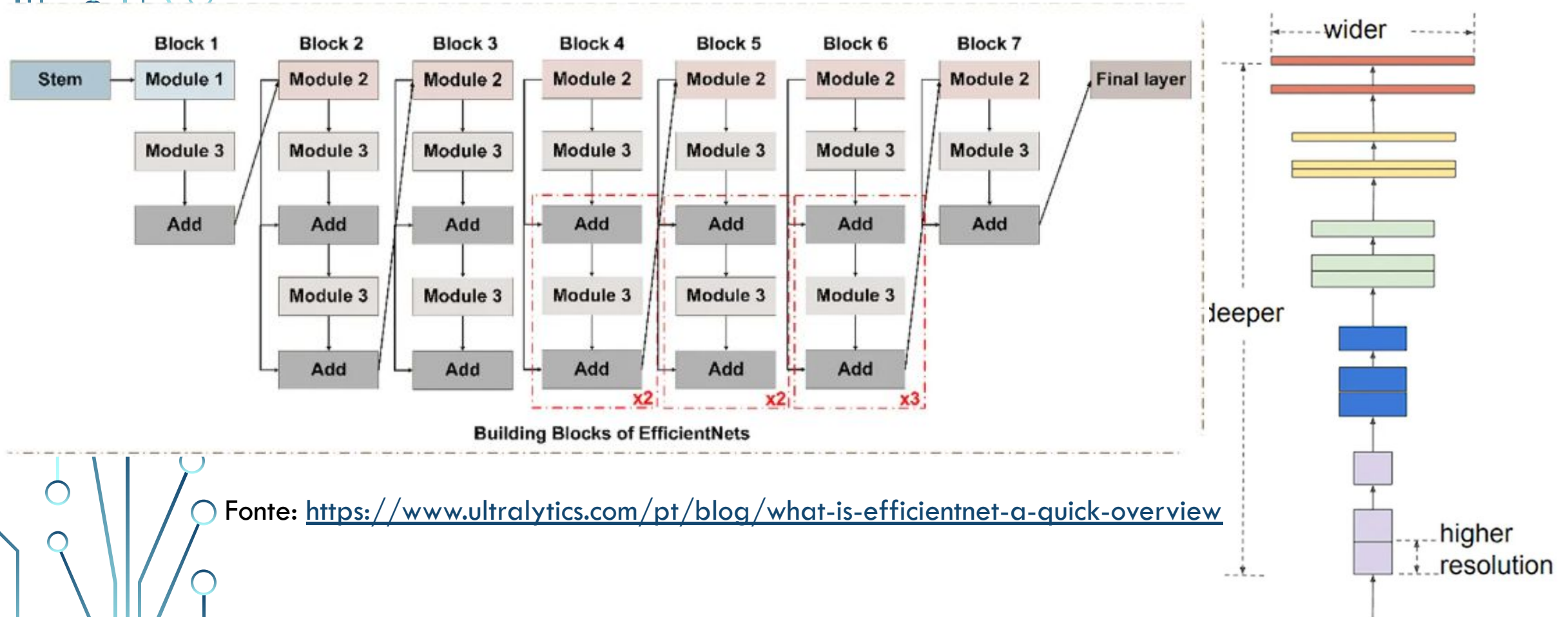


Fig. 8.4.1 Structure of the Inception block.

multiplas CNNs com kernels diferentes, em paralelo, capturando features de diferentes escalas espaciais, em todos níveis de detalhe

REDES CONVOLUCIONAIS -arquiteturas conhecidas

- EfficientNet: dimensionamento composto de resolução, largura e profundidade



Fonte: <https://www.ultralytics.com/pt/blog/what-is-efficientnet-a-quick-overview>

REDES CONVOLUCIONAIS - para que reinventar a roda?

```
from tensorflow import keras
model = keras.applications.resnet50.ResNet50(weights='imagenet')
#download de 98MB (arquivo .h5)
#espera imagens 224x224, outros modelos 299x299 etc.
#suponha que o conjunto de imagens a usar tenha sido escalonado para 0 -
1. Como o ResNet espera 0-255, usa-se o comando:

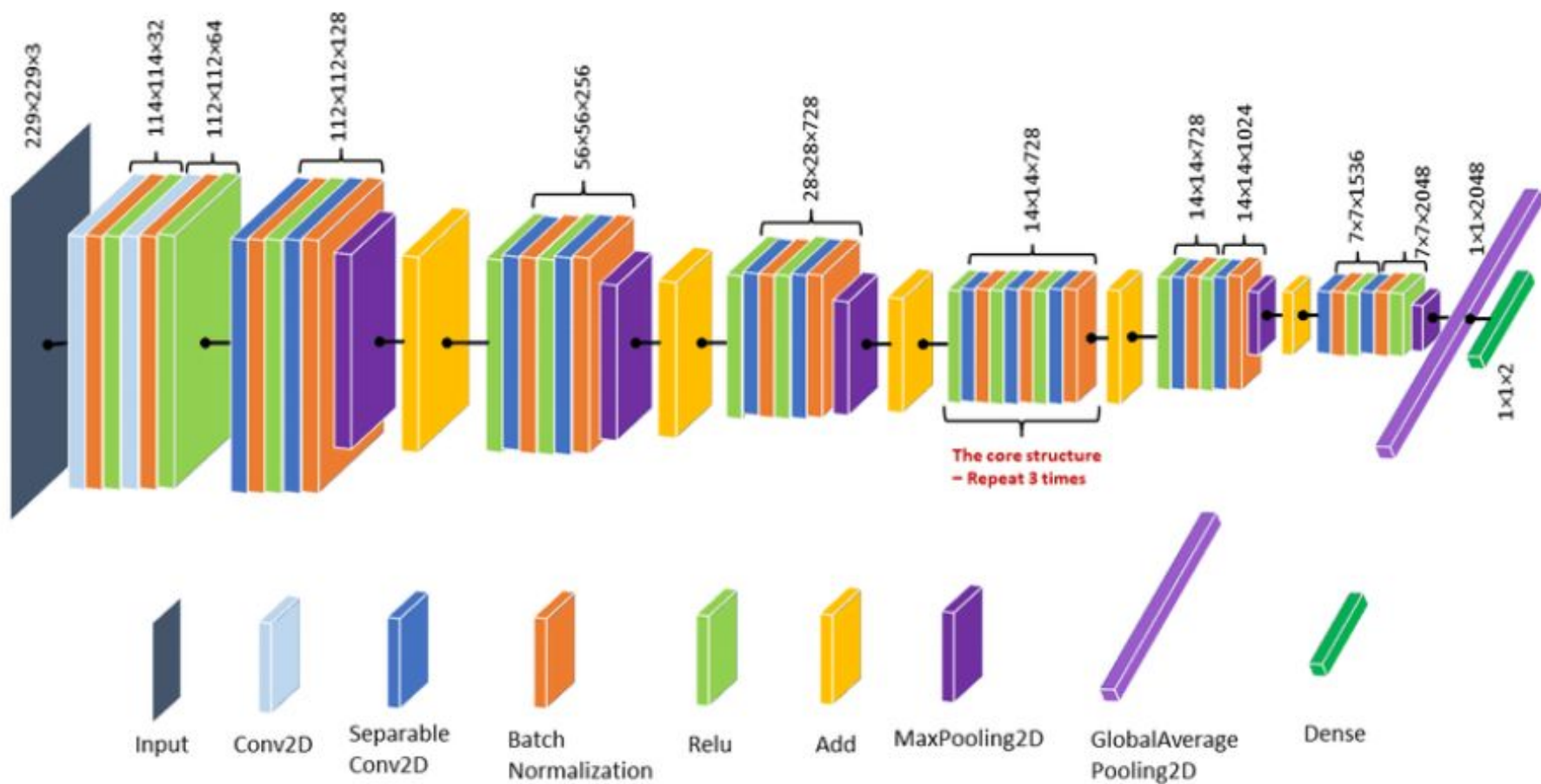
images_resized = tf.image.resize(images, [224,224]) #ou crop_and_resize
inputs = keras.applications.resnet50.preprocess_input(images_resized*255)
Y_proba = model.predict(inputs)
```


REDES CONVOLUCIONAIS - para que reinventar a roda?

```
top_K = keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print('image #{}'.format(image_index))
    for class_id, name, y_proba in top_K[image_index]:
        print(' {} - {:12s} {:.2f}%'.format(class_id, name, y_proba * 100))
```

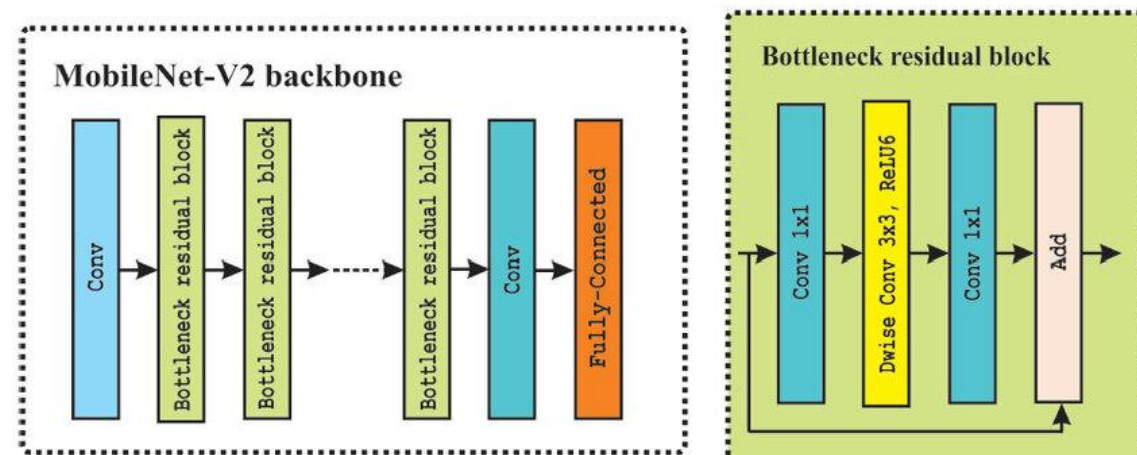
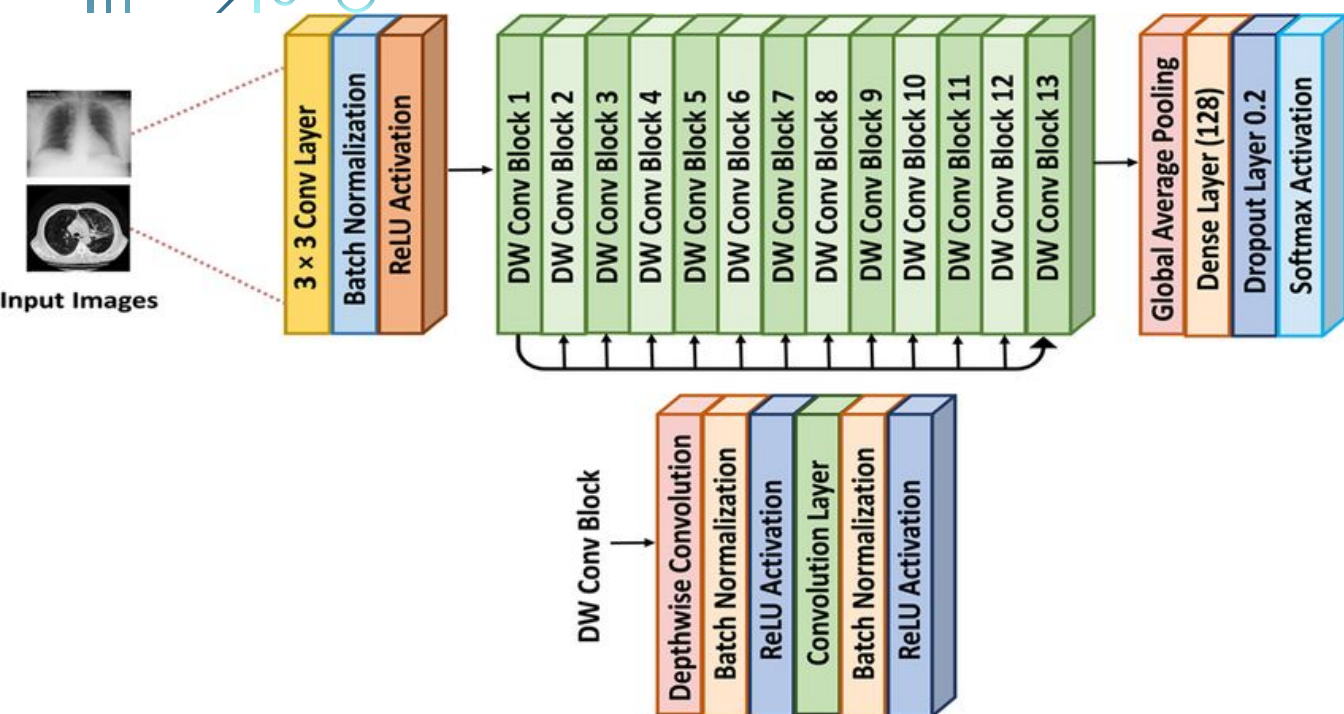
REDES CONVOLUCIONAIS - para que reinventar a roda?

- Xception



REDES CONVOLUCIONAIS - para que reinventar a roda?

- MobileNet (No Keras, V1, V2, V3) Convoluções separáveis em profundidade

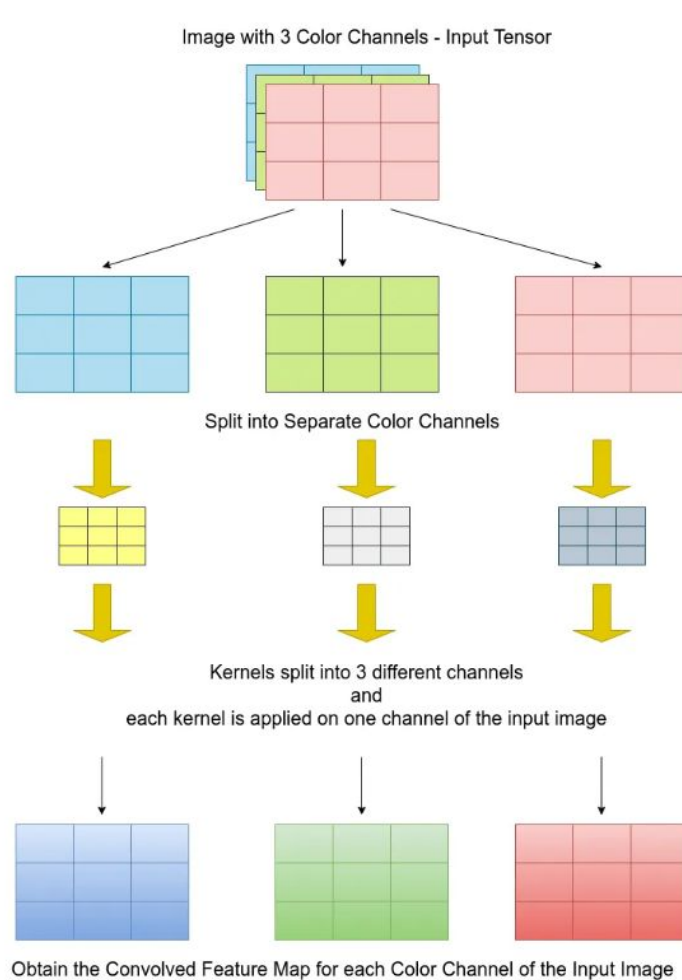


Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogLeNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

Tabela 1. Comparação dos parâmetros e da precisão do MobileNet com o GoogLeNet e o VGG 16 (Fonte: Tabela do artigo original)

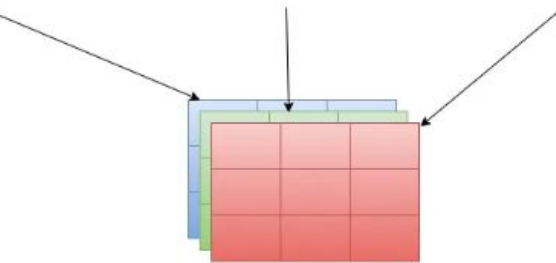
REDES CONVOLUCIONAIS - para que reinventar a roda?

- MobileNet (No Keras, V1, V2, V3) Convoluções separáveis em profundidade

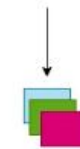


Obtain the Convolved Feature Map for each Color Channel of the Input Image

Obtain the Convolved Feature Map for each Color Channel of the Input Image



Stacking the convolved feature maps to get the Final Output Tensor of the Original 3 Channel Image



Combining the three channels to form n number of channels
This example - three channels are combined to form an output tensor of 1 channel

REDES CONVOLUCIONAIS - para que reinventar a roda?

- [MobileNet](#) (No [Keras](#), V1, V2, V3) Convoluções separáveis em profundidade

Vamos supor que temos um tensor de entrada de tamanho $8 \times 8 \times 3$, E o tensor de saída desejado tem o tamanho $8 \times 8 \times 256$

Em convoluções 2D — Número de multiplicações necessárias — $(8 \times 8) \times (5 \times 5 \times 3) \times (256) = 1.228.800$

Em convoluções separáveis em profundidade — O número de multiplicações necessárias:

a. **Filtragem** — Dividido em canais individuais, portanto, um filtro $5 \times 5 \times 1$ é necessário em vez de um $5 \times 5 \times 3$, e como existem três canais, o número total de filtros $5 \times 5 \times 1$ necessários é 3, então, $(8 \times 8) \times (5 \times 5 \times 1) \times (3) = 3800$

b. **Combinação** — O número total de canais necessários é 256, portanto, $(8 \times 8) \times (1 \times 1 \times 3) \times (256) = 49.152$

Número total de multiplicações = $3.800 + 49.152 = 53.952$

REDES CONVOLUCIONAIS - para que reinventar a roda?

- Carregar modelo Xception, pré-treinado no ImageNet, excluindo as principais redes com `include_top = False`. Ele remove a camada média de pooling global e a camada de saída densa. Depois adicionaremos nossa própria camada de pooling global, com base na saída do modelo básico, seguida por camada densa de saída com softmax

```
#base tf_flowers - import tensorflow_datasets as tfds
#pre-processamento omitido
base_model = keras.applications.xception.Xception(weights='imagenet',
include_top = False)
avg = keras.layers.GlobalAveragePooling2D()(base_model.output)
output = keras.layers.Dense(n_classes,activation='softmax')(avg)
model = keras.Model(inputs=base_model.input, outputs=output)
```

```
# é uma boa técnica a menos no início do treinamento congelar os pesos
das camadas pré-treinadas
for layer in base_model.layers:
    layer.trainable = False
```

REDES CONVOLUCIONAIS - para que reinventar a roda?

```
optimizer = keras.optimizer.SGD(lr=0.2, momentum=0.9, decay=0.01)
model.compile(loss='sparse_categorical_crossentropy',
optimizer=optimizer, metrics=['accuracy'])
history = model.fit(train_set, epochs=5, validation_data=valid_set)
```

#Use GPU

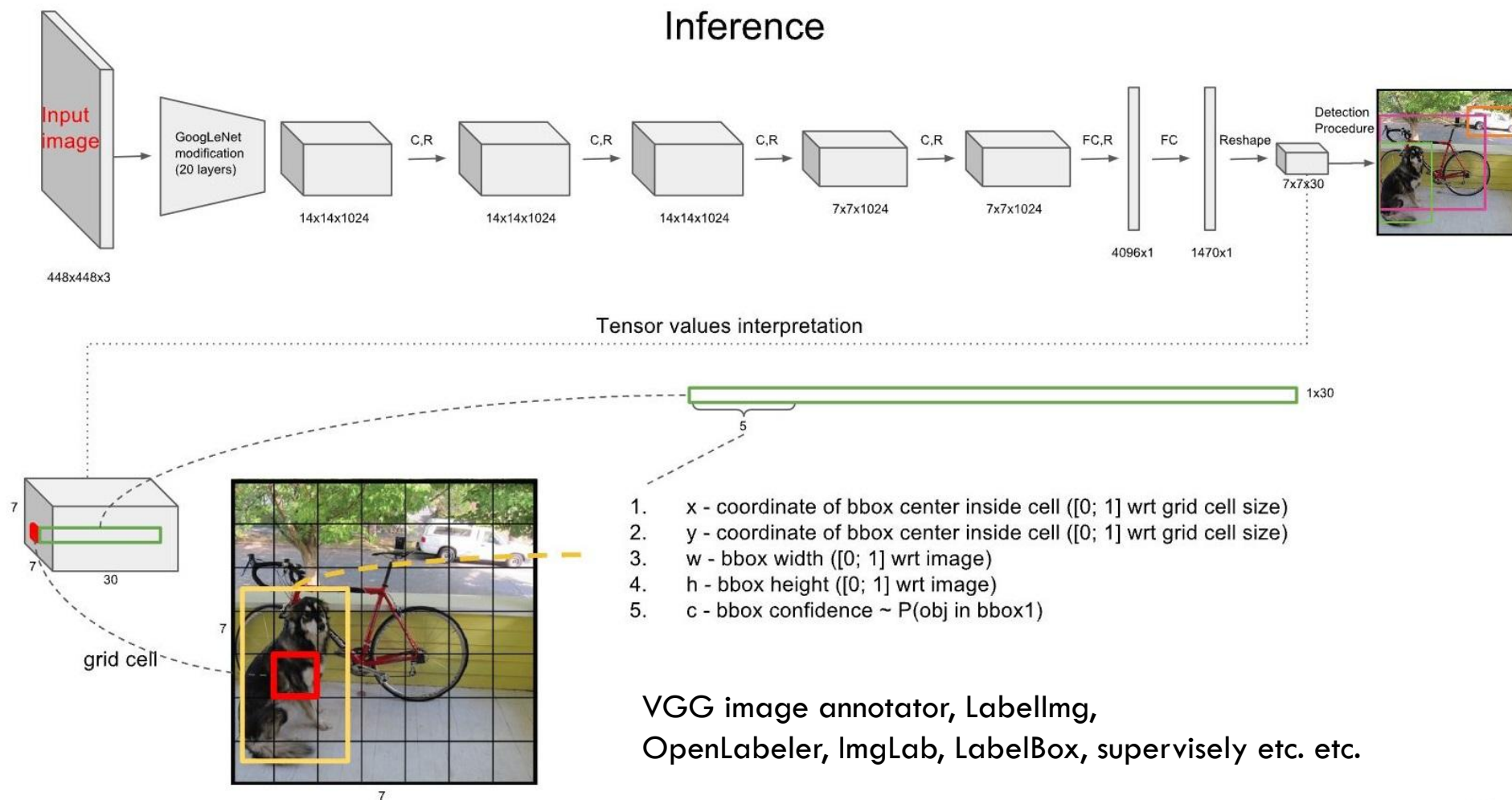
#após algumas épocas, a val_acc deve estar em 75-80%. Significa que as superiores estão bem treinadas. Vamos descongelar todas as camadas, compilar e retreinar

```
optimizer = keras.optimizer.SGD(lr=0.01, momentum=0.9, decay=0.001)
model.compile(...)
model.fit(...)
```

REDES CONVOLUCIONAIS - DETECÇÃO

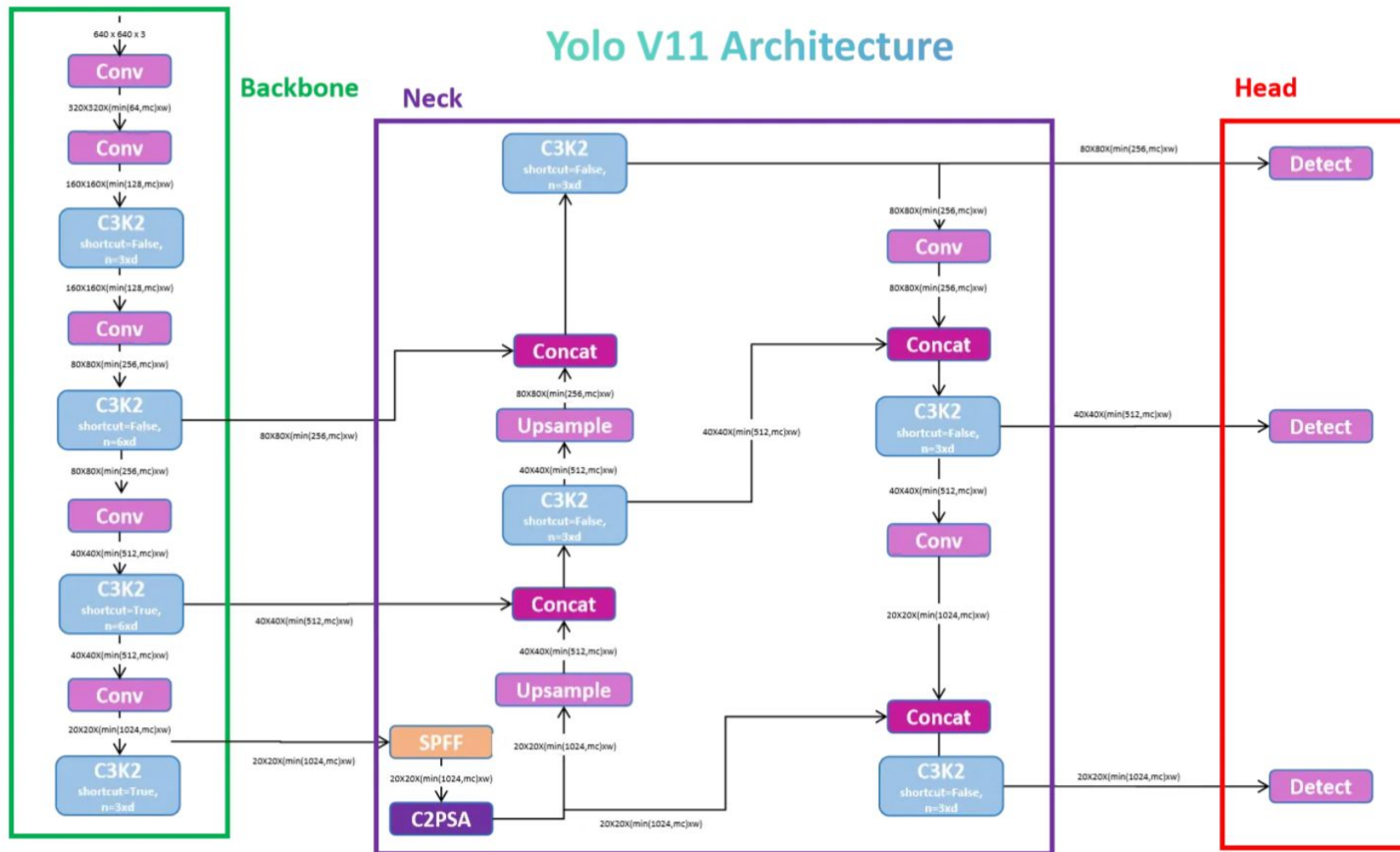
```
base_model = keras.applications.xception.Xception(weights='imagenet',  
                                                    include_top = False)  
avg = keras.layers.GlobalAveragePooling2D()(base_model.output)  
class_output = keras.layers.Dense(n_classes, activation='softmax')(avg)  
loc_output = keras.layers.Dense(4)(avg) #coord hor, ver do centro do  
objeto, altura, largura (4 números)  
  
model = keras.Model(inputs=base_model.input, outputs=[class_output,  
                                                       loc_output])  
  
optimizer = keras.optimizer.SGD(lr=0.2, momentum=0.9, decay=0.01)  
model.compile(loss=['sparse_categorical_crossentropy', 'mse'],  
              loss_weights=[0.8, 0.2], optimizer=optimizer, metrics=['accuracy'])  
  
history = model.fit(train_set, epochs=5, validation_data=valid_set)
```

REDES CONVOLUCIONAIS - DETECÇÃO - YOLO (2015)



VGG image annotator, LabelImg,
OpenLabeler, ImgLab, LabelBox, supervisely etc. etc.

REDES CONVOLUCIONAIS - DETECÇÃO - YOLO



REDES CONVOLUCIONAIS - DETECÇÃO - DEEPPFACE

ANÁLISE DE ATRIBUTO FACIAL (IDADE, EMOÇÃO...) wrapper

